Counting 4-Cycles in Fully Dynamic Graphs: A Cyclic-Join Perspective

Anonymous Author(s)

Abstract

Counting subgraphs in a fully dynamic host graph, viewed through the database lens of incremental view maintenance for cyclic joins, is a primitive whose worst-case update complexity is settled for two of the three patterns on at most four vertices: triangles are tight at $O(\sqrt{m})$ and 4-cliques are tight at $O(m)$. For 4-cycles the long-standing upper bound was $O(m^{2/3})$, inherited from a wedge-maintenance argument of [4]. Recent work of Assadi and Shah [3] breaks this barrier and obtains worst-case update time $O(m^{2/3-\varepsilon})$ for a constant $\varepsilon > 0$, with the speedup driven by fast rectangular matrix multiplication—a tool previously thought inapplicable to dynamic IVM. This essay gives a self-contained treatment of four ingredients of that argument: the reduction from general to 4-layered graphs, a simple $O(n)$ wedge-maintenance baseline, a warm-up algorithm with two frozen relations whose rectangular FMM step is laid out in full, and a closed-form certificate that the resulting constraint system is feasible at $\omega = 2$. An empirical complement benchmarks the simple algorithm against naive recomputation.

CCS Concepts

• **Theory of computation** → **Dynamic graph algorithms**; *Streaming, sublinear and near linear time algorithms*; • **Information systems** → Database query processing.

Keywords

fully dynamic algorithms, subgraph counting, 4-cycles, fast matrix multiplication, incremental view maintenance, cyclic joins

1 Introduction

1.1 Subgraph counting and dynamic graphs

Counting occurrences of a small pattern graph—triangles, 4-cycles, 4-cliques—inside a larger host graph is one of the oldest and most broadly applicable problems in algorithmic graph theory. The same counting primitive underpins clustering and community-detection heuristics in social-network analysis [12], motif-discovery pipelines in computational biology [8], and recurring subexpression evaluation in database query processing [11]. In each of these settings the host graph is not a static input but an object that evolves over time as edges are added and removed.

The *fully dynamic* model formalises this regime: the host graph G receives a sequence of edge insertions and deletions, and after each update the algorithm must report the current number of occurrences of the fixed pattern. The relevant complexity measure is the *worst-case update time*, the maximum work spent on any single update over the entire stream—a strictly stronger guarantee than amortised time, and the one we will track throughout this essay.

FIG 1 PLACEHOLDER

Figure 1: A 4-layered graph encoding a cyclic join $A \bowtie B \bowtie C \bowtie D$. One layered 4-cycle highlighted. Adapted from [3], Fig. 1.

1.2 The cyclic-join framing

Subgraph counting also has a natural reading in database theory. Given four binary relations $A(L_1, L_2), B(L_2, L_3), C(L_3, L_4), D(L_4, L_1)$, the size of the cyclic four-way join $J = A \bowtie B \bowtie C \bowtie D$ counts exactly the number of 4-cycles in the 4-layered graph whose layers are the attribute domains $L_1, \ldots, L_4$ and whose edges encode the relation tuples [9, 10]. Figure 1 illustrates the correspondence on a small instance. Maintaining $|J|$ under tuple insertions and deletions is the *incremental view-maintenance* (IVM) problem for this cyclic join, and the dynamic 4-cycle counting question is, up to a structural reduction, the same problem. We make this equivalence precise—and lift it from the layered setting to general graphs—in Section 3; Theorem 1 is the key statement.

1.3 The state of the art

For each pattern, the goal is to express the worst-case update time as a function of m, the current number of edges. The state of the art forms a small ladder, summarised in Table 1. Triangles are essentially settled: an upper bound of $O(\sqrt{m})$ is due to Kara, Ngo, Nikolic, Olteanu, and Zhang [6], matched up to subpolynomial factors by an $\Omega(m^{1/2-\gamma})$ conditional lower bound under the OMv conjecture of Henzinger, Krinninger, Nanongkai, and Saranurak [5]. At the other end of the ladder, 4-cliques are pinned to $O(m)$ for combinatorial algorithms, where a folklore upper bound meets the $\Omega(m^{1-\gamma})$ combinatorial 4-clique-conjecture lower bound of Hanauer, Henzinger, and Hua [4]. Sandwiched between these is the 4-cycle. The same authors of [4] obtained a worst-case update time of $O(m^{2/3})$ by generalising the wedge-maintenance idea behind the triangle bound, while the only matching lower bound remains the OMv-based $\Omega(m^{1/2-\gamma})$ inherited from the triangle case. The $m^{1/2}$–$m^{2/3}$ gap is the empty rung in the ladder, and asks an obvious question: is $O(m^{2/3})$ the right answer, or can FMM-based techniques push the upper bound below it?

1.4 The Assadi–Shah result

The answer, due to Assadi and Shah [3], is that $O(m^{2/3})$ is *not* the right answer: they give an algorithm with worst-case update time

[1] $\varepsilon > 0$ is a constant depending on the matrix-multiplication exponent ω; see Theorem 6.

Pattern	Upper bound	Lower bound
3-cycle (triangle)	$O(\sqrt{m})$ [6]	$\Omega(m^{1/2-\gamma})$ [5]
4-cycle	$O(m^{2/3-\varepsilon})$ [3][1]	$\Omega(m^{1/2-\gamma})$ [5]
4-clique	$O(m)$ folklore	$\Omega(m^{1-\gamma})$ [4]

Table 1: State of the art for fully dynamic subgraph counting. Lower bounds are conditional (OMv for 3- and 4-cycles; combinatorial 4-clique conjecture for 4-cliques).

$O(m^{2/3-\varepsilon})$ for some constant $\varepsilon > 0$; we restate this as Theorem 6. The constant ε depends on the matrix-multiplication exponent ω. With the current best $\omega < 2.371339$ [1] the bound yields $\varepsilon \approx 0.0098$; under the conjectural optimum $\omega = 2$ it improves to $\varepsilon = 1/24$.

Two features of this result make it conceptually surprising. First, the speedup engine is fast (rectangular) matrix multiplication, and it was widely believed that FMM cannot help in dynamic IVM at all: Strassen-style algorithms [13] require both factors to be available up front, whereas in the dynamic setting the matrices themselves are the changing objects. Static FMM-based subgraph counting has been known since [2], but no analogous dynamic speedup had appeared. Second, this surprise is sharp: the algorithm of [3] only beats $O(m^{2/3})$ once $\omega < 5/2 - O(\varepsilon)$, so even Strassen's $\omega \approx 2.81$ is insufficient and only the modern bounds of [1] push the exponent below the threshold. The contrast with 4-cliques is instructive: their $\Omega(m^{1-\gamma})$ lower bound rests on the *combinatorial* 4-clique conjecture and therefore blocks FMM-based speedups, while the $\Omega(m^{1/2-\gamma})$ lower bound for 4-cycles only rules out combinatorial reductions through OMv and leaves room for an algebraic improvement of exactly the kind delivered here.

1.5 What this essay does

After notation in Section 2, the body of this essay delivers four formal results. Section 3 gives a self-contained reduction from the general-graph problem to the 4-layered version, culminating in an update-protocol argument (Theorem 1) that turns counted 3-walks into 3-paths. Section 4 presents and fully proves the simple $O(n)$ wedge-maintenance baseline (Lemma 2), which doubles as the algorithm we benchmark. Section 5 develops the warm-up algorithm in which A and C are frozen, gives a complete dimension analysis of its rectangular FMM step (Theorem 4), and provides a closed-form verification that the constraint system admits a positive ε_1 at $\omega = 2$ (Lemma 5). An empirical study of the simple algorithm against naive recomputation appears in Section 6. We sketch how the warm-up lifts to Theorem 6 via phases but deliberately leave the de-assumption machinery—tiny vertices and vertex-class transitions—out of scope, and we do *not* reprove Theorem 6.

2 Preliminaries

2.1 Graphs and subgraph notation

Throughout, $G = (V, E)$ is a simple, undirected, unweighted graph with no self-loops or parallel edges. We write $n = |V|$ and $m = |E|$. For a vertex $v \in V$, $\deg(v)$ denotes its degree and $N(v) = \{u : \{u, v\} \in E\}$ its neighborhood.

A *k-path* is a sequence of $k+1$ distinct vertices $(v_0, v_1, \ldots, v_k)$ with $\{v_i, v_{i+1}\} \in E$ for all $0 \le i < k$. A 2-path is also called a *wedge*. A *k-cycle* is a sequence $(v_0, \ldots, v_{k-1})$ of k distinct vertices with $\{v_i, v_{i+1 \bmod k}\} \in E$ for all $0 \le i < k$.

A *4-layered graph* is a graph whose vertex set partitions as $V = L_1 \cup L_2 \cup L_3 \cup L_4$, with each L_i an independent set and edges only between consecutive layers L_i, L_{i+1} (and between L_4 and L_1, closing the cycle). The four edge sets are encoded by biadjacency matrices $A \subseteq L_1 \times L_2$, $B \subseteq L_2 \times L_3$, $C \subseteq L_3 \times L_4$, and $D \subseteq L_4 \times L_1$. A *layered k-path* (resp. *layered k-cycle*) is a k-path (resp. k-cycle) with one vertex in each consecutive layer. We index layers modulo 4 throughout.

2.2 Dynamic graph model

The graph evolves under a sequence of updates $u_1, u_2, \ldots$, where each u_i is either INSERT(e) or DELETE(e) for an edge e on a fixed vertex set. Let G_i be the graph after the i-th update, with G_0 the empty graph. The model is *fully dynamic* because both insertions and deletions are permitted. Immediately after each update, the algorithm must output the exact number of 4-cycles in G_i.

The complexity measure is the *worst-case update time*: the maximum time spent processing any single update, taken over every update index and every legal sequence. This is strictly stronger than *amortized* time, which bounds only the average over a sequence; the headline bound of [3] is worst-case.

2.3 Fast matrix multiplication

Let ω denote the exponent of square matrix multiplication: two $n \times n$ matrices can be multiplied in $O(n^\omega)$ arithmetic operations. Strassen [13] established $\omega < 3$, and the current record is $\omega < 2.371339$ [1]. It is a long-standing open problem whether $\omega = 2$.

For rectangular products we write $\omega(a, b, c)$ for the exponent of multiplying an $n^a \times n^b$ matrix by an $n^b \times n^c$ matrix in $n^{\omega(a,b,c)}$ time, so $\omega(1, 1, 1) = \omega$. Improved upper bounds for $\omega(a, b, c)$ across the relevant asymmetric range are also given in [1]; the warm-up algorithm of Section 5 relies on these. An algorithm is *combinatorial* if it does not exploit Strassen-style algebraic identities; combinatorial algorithms therefore do not benefit from $\omega < 3$.

2.4 The OMv conjecture

The *Online Matrix-Vector multiplication (OMv) conjecture* of [5] concerns the following problem: an algorithm preprocesses an $n \times n$ Boolean matrix M in polynomial time, then receives n Boolean vectors $v_1, \ldots, v_n$ one at a time and must output Mv_i before seeing v_{i+1}. The conjecture asserts that no algorithm solves the entire instance in total time $O(n^{3-\gamma})$ for any constant $\gamma > 0$.

Reductions from OMv yield conditional $\Omega(m^{1/2-\gamma})$ lower bounds on the worst-case update time for dynamically maintaining the number of triangles or 4-cycles [4, 5]. These bounds hold even against algorithms that invoke fast matrix multiplication. The contrast with 4-cliques is instructive: the prevailing $\Omega(m^{1-\gamma})$ lower bound for 4-cliques relies on the *combinatorial* 4-clique conjecture and therefore does not constrain non-combinatorial algorithms [4]. For 4-cycles the OMv-based barrier sits well below the best known upper bound, and the result we discuss in Section 5 narrows—but does not close—this gap.

3 The cyclic-join equivalence

3.1 Joins as layered graphs

The motivating object behind 4-cycle counting on a 4-layered graph is the cyclic four-way join

$$J = A(L_1, L_2) \bowtie B(L_2, L_3) \bowtie C(L_3, L_4) \bowtie D(L_4, L_1),$$

familiar in database theory as a fundamental hard case for join evaluation [9, 10]. The four binary relations A, B, C, D over the attribute domains $L_1, \ldots, L_4$ are exactly the biadjacency matrices of Section 2.1: a tuple in the relation between L_i and L_{i+1} corresponds to an edge between the two consecutive layers in $G' = (L_1 \cup L_2 \cup L_3 \cup L_4, E')$. A satisfying assignment $(a, b, c, d) \in L_1 \times L_2 \times L_3 \times L_4$ of J is precisely a closed sequence $a \to b \to c \to d \to a$ that visits one vertex per layer, that is, a layered 4-cycle in G'. The size of the join therefore equals the number of layered 4-cycles in G'. Figure 1 illustrates the correspondence on a small instance. Maintaining $|J|$ under tuple updates is then exactly the incremental view-maintenance problem for this cyclic join, which is the database-theoretic reading of our counting task.

3.2 Lifting a general graph to a 4-layered graph

From a general graph $G = (V, E)$, build a 4-layered graph $G' = (V', E')$ by replicating V in each layer: $V' = V \times \{1, 2, 3, 4\}$, with the i-th copy of V playing the role of L_i. Every edge $\{u, v\} \in E$ is lifted into all four biadjacency matrices: each of A, B, C, D records the pair (u, v) (equivalently, each layer pair $L_i \to L_{(i \bmod 4)+1}$ gains the two directed edges (u_i, v_{i+1}) and (v_i, u_{i+1}) that encode the undirected edge between adjacent-layer copies). The construction is depicted in Figure 2.

Crucially, a single update (u, v) in G becomes four updates in G', one per matrix, and the order in which these are applied is load-bearing for the proof of Theorem 1. We adopt the following *update protocol* [3]:

- **Insertion.** Insert (u, v) first into D, then into C, then into B, and finally into A.
- **Deletion.** Delete (u, v) from A first, then from B, then from C, and last from D.
- **Query.** Read off the change in the layered 4-cycle count immediately after the D-update – between the D step and the next of A, B, C.

In both cases, the protocol guarantees that the lifted edge (u, v) is *absent* from A, B, and C at the moment of query: on an insertion the edge has not yet propagated past D, and on a deletion it has already been removed from A, B, C before D. This asymmetry between D and the other three matrices is exactly what forces every length-3 *walk* counted by the algorithm to be a length-3 *path* in the underlying graph; without it, a walk could revisit u or v via the freshly written edge and the count would over-report.

3.3 Equivalence theorem

Theorem 1 (General–layered equivalence)**.** *Let $\{u, v\}$ be the edge currently being updated in G, and let G' be the lifted graph of Section 3.2 immediately after the D-step of the protocol (and before any subsequent A, B, C step). At this moment, the number of length-3 walks in G' from $(u, 1) \in L_1$ to $(v, 4) \in L_4$ equals the number of length-3 paths from u to v in the current G.*

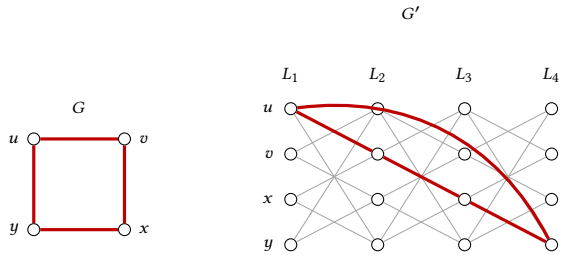

Figure 2: Lifting a general graph G (left) to a 4-layered graph G' (right). Every vertex of V is replicated in each layer $L_1, \ldots, L_4$; gray edges schematically depict the lifted copies between consecutive layers. The 4-cycle u, v, x, y in G corresponds to the highlighted layered 4-cycle $(u, 1) \to (v, 2) \to (x, 3) \to (y, 4) \to (u, 1)$, with the closing $L_4 \to L_1$ edge drawn as a curved arc. For legibility, only the highlighted cycle is shown in full; remaining lifted edges are partial.

Proof. We follow the argument of [3]. The forward direction is immediate: every length-3 path u, x, y, v in G has, by construction, a corresponding lifted walk $(u, 1), (x, 2), (y, 3), (v, 4)$ in G' whose three edges are the lifts of the three edges of the original path into the matrices A, B, and C respectively. Distinct paths in G produce distinct lifted walks in G', so the count of length-3 paths from u to v in G is at most the count of length-3 walks from $(u, 1) \in L_1$ to $(v, 4) \in L_4$ in G'.

For the reverse direction, fix a length-3 walk $(u, 1), (x, 2), (y, 3), (v, 4)$ in G' and write its underlying sequence in V as u, x, y, v. We show that the four vertices are distinct, so that the walk lifts a path in G. Because G contains no self-loops, the edge $\{w, w\}$ is never in E and therefore never lifted; this rules out $x = u$, $x = y$, and $y = v$ (each of which would require a self-loop in G to be lifted into A, B, or C). The remaining inequalities involve the edge (u, v) itself. If $x = v$, the first edge of the walk would be the lift of $\{u, v\}$ into A; if $y = u$, the third edge would be the lift of $\{u, v\}$ into C. But the protocol guarantees that immediately after the D-update, (u, v) is absent from each of A, B, C, so neither lifted edge exists in G' at query time, contradiction. Hence u, x, y, v are four distinct vertices and the walk lifts a length-3 path in G. Combined with the (clearly injective) forward map, this establishes a bijection between length-3 paths in G and length-3 walks in G' from $(u, 1)$ to $(v, 4)$. □

3.4 Consequence

Theorem 1 reduces the dynamic 4-cycle counting problem on a general graph to dynamically counting layered 4-cycles on a 4-layered graph: the number of 4-cycles through a new edge (u, v) is exactly the layered 3-walk count delivered by the lifted instance under the protocol of Section 3.2, and the lift incurs only a constant blow-up in the number of layered updates per general-graph update. The warm-up algorithm of Section 5 therefore works directly on a 4-layered graph with biadjacency matrices A, B, C, D. The simple $O(n)$ baseline of Section 4 is presented in the original general-graph

Algorithm 1 Maintain 4-cycle count via wedge matrix W.

```
1:  procedure Insert(u, v)
2:      Δ ← Σ_{w∈N(u)} W[w, v]                          ▷ query
3:      C ← C + Δ
4:      for w ∈ N(u) do W[w, v] += 1; W[v, w] += 1
5:      end for
6:      for w ∈ N(v) do W[w, u] += 1; W[u, w] += 1
7:      end for
8:      add (u, v) to E
9:  end procedure
10: procedure Delete(u, v)
11:     remove (u, v) from E
12:     for w ∈ N(u) do W[w, v] −= 1; W[v, w] −= 1
13:     end for
14:     for w ∈ N(v) do W[w, u] −= 1; W[u, w] −= 1
15:     end for
16:     Δ ← Σ_{w∈N(u)} W[w, v]                          ▷ query
17:     C ← C − Δ
18: end procedure
```

setting and uses the same query-then-update timing idea adapted to its single wedge matrix.

4 A simple $O(n)$ algorithm

4.1 The wedge-counting idea

The simple algorithm of this section operates directly on a general graph $G = (V, E)$ on n vertices, without invoking the layered reduction of Section 3. The starting observation is that an edge update only changes the 4-cycle count through cycles *containing the updated edge*: if the count before an insertion of (u, v) is C_{old}, the count afterwards is $C_{\text{old}} + \Delta$, where Δ is the number of 4-cycles in the new graph that use the edge (u, v). Each such 4-cycle is the closure $u \to w \to x \to v \to u$ of a length-3 path from u to v in the graph $G \setminus \{(u, v)\}$ (with the edge removed), so

$$\Delta \;=\; \#\{\text{ length-3 paths from } u \text{ to } v \text{ in } G\,\}.$$

A length-3 path decomposes as a first edge $u \to w$ with $w \in N(u)$ followed by a length-2 path (a *wedge*) from w to v. Let $W[w, v]$ denote the number of wedges between w and v in G. Then

$$\Delta \;=\; \sum_{w \in N(u)} W[w, v]. \tag{1}$$

In principle the right-hand side counts length-3 *walks* u, w, x, v rather than paths; the order-of-operations rule of Section 4.2 guarantees that at query time the edge (u, v) is absent from W, which forces every counted walk to be a path with four distinct vertices and makes identity (1) hold without any correction term. The algorithm therefore reduces 4-cycle maintenance to maintaining the wedge-count matrix W.

4.2 The algorithm

Algorithm 1 maintains a global counter C for the current 4-cycle count alongside the wedge matrix W. Each update performs two distinct kinds of work: a *query* that uses identity (1) to compute the cycles through the updated edge, and a *maintenance* step that adjusts W to reflect the new edge set. Both steps touch only the rows and columns indexed by neighbours of u and v.

The order in which these two steps are performed is load-bearing, and is the general-graph specialisation of the update protocol of Section 3.2: in both settings the new edge must be *absent* from the relevant data structure at the moment of query. For correctness, W must not contain any wedge that uses the edge (u, v) at the moment the query is evaluated; otherwise the expression $\sum_{w \in N(u)} W[w, v]$ would count walks that revisit u or v, breaking identity (1). On an Insert, the edge (u, v) is not yet in E when the procedure starts, so the query is evaluated *first*, and only afterwards is W updated to record the wedges newly created through the inserted edge. On a Delete, the edge (u, v) is still in E at the start, so W is updated *first*—removing the wedges that used (u, v)—and the query is evaluated only once those wedges are gone. In both cases, W at query time encodes only wedges of the graph $G \setminus \{(u, v)\}$, which is what identity (1) requires.

4.3 Correctness and complexity

Lemma 2 (Correctness and complexity of Algorithm 1)**.** *Algorithm 1 maintains the exact 4-cycle count of any fully dynamic graph in worst-case update time $O(n)$.*

Proof. We argue maintenance and counting separately [3].

(a) Maintenance. Each update modifies only the entries $W[w, v]$ and $W[v, w]$ for $w \in N(u)$, and $W[w, u]$ and $W[u, w]$ for $w \in N(v)$. The four loops in Algorithm 1 therefore touch at most $|N(u)| + |N(v)| \le 2n$ entries of W, each in constant time, so the maintenance step costs $O(n)$ per update.

(b) Counting through the new edge. The query computes

$$\Delta \;=\; \sum_{w \in N(u)} W[w, v] \;=\; \sum_{w \in N(u)} \; \sum_{x \,:\, w\text{-}x\text{-}v \text{ a wedge}} 1,$$

where the inner sum ranges over wedge midpoints x between w and v. This is the number of *walks* u, w, x, v in the graph recorded by W at query time. We must show that every such walk is in fact a path, i.e. that u, w, x, v are four distinct vertices.

Note first that $u \neq v$ since the update concerns a non-loop edge. We must rule out the five remaining equalities $w \in \{u, v\}$ and $x \in \{u, v, w\}$. Three of these are excluded by simplicity of G: $w \neq u$ because $w \in N(u)$, and $x \neq w$ and $x \neq v$ because $W[w, v]$ counts wedges w–x–v with two distinct edges (no self-loops). The remaining two—$w \neq v$ and $x \neq u$—both rely on the fact that $(u, v) \notin E$ at query time. This holds in both update modes: on an insertion the edge has not yet been added, while on a deletion it has already been removed. Consequently $v \notin N(u)$, which forces $w \neq v$, and any wedge of the form w–u–v would require the absent edge $(u, v) \in E$, so it cannot contribute to $W[w, v]$ at query time, ruling out $x = u$.

All four vertices are therefore distinct, so each walk counted by Δ is a length-3 path from u to v. Conversely, every length-3 path u, w, x, v in the query-time graph contributes exactly once: it appears in the unique summand of identity (1) indexed by its first hop w, in which the wedge w–x–v is one of the $W[w, v]$ counted configurations. The map between length-3 paths and counted walks is therefore a bijection, and Δ is the correct increment to the 4-cycle count.

Combining (a) and (b), each update performs $O(n)$ work for maintenance and $O(|N(u)|) = O(n)$ work for the query, for a total worst-case update time of $O(n)$. □

4.4 Why $O(n)$ is not enough

The $O(n)$ bound of Lemma 2 is tight for sparse graphs in the worst case: when an update touches a vertex of degree $\Theta(n)$, every entry of the corresponding row of W may need to be inspected. For sparse graphs with $m = \Theta(n)$ this matches the natural target of expressing the update time purely as a function of m, since $n = \Theta(m)$. For denser graphs the bound is loose: when $n^2 = \omega(m)$, an update time of $O(n)$ can be much larger than any reasonable function of m, and the algorithm performs work that is not justified by the size of the current graph. A second, practical, drawback is the memory footprint: the wedge matrix W has n^2 entries, which is manageable for graphs with a few thousand vertices but becomes prohibitive at the scale of modern social and web graphs, regardless of how sparse they are.

These two shortcomings motivate the search for an algorithm whose update time is bounded purely as a function of the current edge count m. The first such bound was the $O(m^{2/3})$ algorithm of [4], which already uses a wedge-based approach as its starting point but partitions vertices by degree to break the $O(n)$ barrier. Section 5 develops the warm-up algorithm of [3], whose update time is $O(m^{2/3-\varepsilon_1})$ under simplifying assumptions and which is the building block for the $O(m^{2/3-\varepsilon})$ result that finally crosses the $m^{2/3}$ threshold.

5 The warm-up algorithm

5.1 Setup and assumptions

The warm-up algorithm of [3] solves a restricted version of the layered counting problem in which two of the four biadjacency matrices are frozen. Concretely, we assume that the edge updates affect only B and D, while A and C are fixed in advance. By symmetry it suffices to handle the query case in which the latest update is to D and we count new 4-cycles by counting 3-paths from $u \in L_1$ to $v \in L_4$ through A, B, C.

Two further structural assumptions support the analysis. First, the number of vertices in each layer satisfies $n \le m^{2/3+2\varepsilon}$, where $\varepsilon > 0$ is a small constant fixed below; this rules out pathological layers that are much larger than the edge set warrants. Second, vertices do not migrate between degree classes during the algorithm; the bookkeeping needed to lift this assumption is the subject of a separate part of the source paper [3, §6–7] and is out of scope here. Under these assumptions the goal is a worst-case update time of $O(m^{2/3-\varepsilon_1})$ for some constant $\varepsilon_1 > 0$, matching the running time we will need when this algorithm is later invoked as a subroutine in the full result.

5.2 Vertex degree classes

The algorithm partitions vertices in the outer layers L_1 and L_4 into three degree classes, measured against the (fixed) matrices A and C respectively. A vertex is *High* (H) if its degree lies in $[m^{2/3-\varepsilon_1}, n]$, *Medium* (M) if it lies in $[m^{1/3+\varepsilon_1}, 2m^{2/3-\varepsilon_1}]$, and *Low* (L) otherwise; the slight overlap between the M and H ranges keeps class boundaries on a single threshold rather than at an exact cutoff. We use

Class	Stored products
High in L_1, L_4	$A^{H*}\cdot B_{<i}$; $A^{H*}\cdot B_{<i}\cdot C^{*H}$; $B_{<i}\cdot C^{*H}$
Medium in L_1, L_4	$A^{M*}\cdot B_{<i}$; $B_{<i}\cdot C^{*M}$
Low in L_1, L_4	$A^{L*}\cdot B_{<i,DD}$, $A^{L*}\cdot B_{<i,SS}$, $A^{L*}\cdot B_{<i,SD}$, and three symmetric products with C^{*L}

Table 2: Data structures maintained by the warm-up algorithm. Adapted from [3], Table 1.

A^{H*}, A^{M*}, A^{L*} to denote the row-restrictions of A to each class, and symmetrically C^{*H}, C^{*M}, C^{*L} for column-restrictions of C.

Inner-layer vertices in L_2 and L_3 are classified per chunk (§5.3): inside chunk B_i, a vertex is *Sparse* (S) if its degree in B_i is at most $m^{1/3-\varepsilon_2}$ and *Dense* (D) otherwise. We correspondingly write $B_{i,DD}, B_{i,SS}, B_{i,DS}, B_{i,SD}$ for the four submatrices of B_i obtained by restricting both endpoints by class. The previous $O(m^{2/3})$ algorithm [4] used only two outer classes (high and low); the third, medium, class is what makes room for a rectangular FMM product in the warm-up [3], since it lets the dense case ($A^{L*}\cdot B_{i,DD}$) sit on dimensions that current rectangular bounds can exploit.

5.3 Chunks and lazy maintenance

The stream of edge updates to B is grouped into *chunks* $B_1, B_2, \ldots$ of size $m^{2/3-\varepsilon_1}$, and we write $B_{<i} := \sum_{j<i} B_j$ for the aggregate of all completed chunks. Cheap data structures are maintained eagerly: each individual update touches only $O(m^{1/3+\varepsilon_1})$ rows or columns, well within budget. Expensive data structures, the matrix products that require fast matrix multiplication, are not affordable per-update; we instead compute them once per chunk and amortize. Concretely, the products needed for chunk B_i are computed during the $m^{2/3-\varepsilon_1}$ updates of the next chunk B_{i+1}, using a budget of $O(m^{4/3-2\varepsilon_1})$ total work and therefore $O(m^{2/3-\varepsilon_1})$ per update.

While inside chunk B_{i+1}, queries are answered by combining the freshly available data structures for $B_{<i+1}$ with a *lazy evaluation* for the in-progress chunk: edges of B_i and B_{i+1} are scanned one by one against the query endpoints, contributing $O(1)$ time per edge. A subtle issue arises when an edge inserted in chunk B_j is deleted in a later chunk $B_{j'}$; following [3] this is handled by recording the deletion as a *negative* edge in $B_{j'}$, so that the arithmetic in $B_{<i}$ remains correct without retroactively editing earlier chunks.

5.4 Data structures

Table 2 lists the matrix products maintained across the three outer-layer classes, all evaluated against the chunk-aggregate $B_{<i}$. Each such product encodes the count of two- or three-hop paths between the indicated vertex classes. For example, $A^{H*}\cdot B_{<i}$ records, for every High vertex $u \in L_1$ and every $z \in L_3$, the number of L_1–L_2–L_3 wedges from u to z that use only edges of completed chunks. In database terminology this is a select-project-aggregate over the join $A \bowtie B$ restricted to L_1^H. The triple product $A^{H*}\cdot B_{<i}\cdot C^{*H}$ similarly stores the number of 3-paths between any two High endpoints, computed as a single rectangular matrix product per chunk transition. Medium classes require only the one-sided products $A^{M*}\cdot B_{<i}$ and $B_{<i}\cdot C^{*M}$ (the third hop is iterated at query time), while Low

classes need finer S/D decompositions of $B_{<i}$ so that the dense-block product $A^{L*} \cdot B_{<i,DD}$ can be computed once per chunk via FMM.

Lemma 3 (Update-time accounting for Table 2). *The worst-case time to update all data structures in Table 2 is $O(m^{2/3-\varepsilon_1})$ per update, with $O(m^{4/3-2\varepsilon_1})$ amortized over chunk transitions.*

5.5 Worked example: the FMM step

Theorem 4 (Cost of the rectangular FMM step). *The matrix product $A^{L*} \cdot B_{i,DD}$ can be computed in time $m^{\omega(2/3+2\varepsilon,1/3-\varepsilon_1+\varepsilon_2,1/3-\varepsilon_1+\varepsilon_2)}$.*

Proof. We bound the effective dimensions of the two factors. The matrix A^{L*} has at most $|L_1| \le n \le m^{2/3+2\varepsilon}$ rows by Assumption 1, and we restrict its columns to indices in L_2 that are dense within chunk B_i. By definition, a Dense vertex of L_2 has degree at least $m^{1/3-\varepsilon_2}$ in B_i, so the total number of dense vertices in L_2 is at most $|B_i|/m^{1/3-\varepsilon_2} = m^{(2/3-\varepsilon_1)-(1/3-\varepsilon_2)} = m^{1/3-\varepsilon_1+\varepsilon_2}$. Hence the relevant dimensions of A^{L*} are $m^{2/3+2\varepsilon} \times m^{1/3-\varepsilon_1+\varepsilon_2}$. The factor $B_{i,DD}$ is the restriction of B_i to dense rows and dense columns, so by the same accounting it has dimensions $m^{1/3-\varepsilon_1+\varepsilon_2} \times m^{1/3-\varepsilon_1+\varepsilon_2}$. The product is therefore a rectangular matrix multiplication of shape $(2/3+2\varepsilon,\ 1/3-\varepsilon_1+\varepsilon_2,\ 1/3-\varepsilon_1+\varepsilon_2)$, which can be performed in time $m^{\omega(2/3+2\varepsilon,1/3-\varepsilon_1+\varepsilon_2,1/3-\varepsilon_1+\varepsilon_2)}$ by definition of the rectangular FMM exponent [1]. □

This is the only step in the warm-up that genuinely uses fast matrix multiplication, and it is the step that constrains the parameters. Charging the cost against the per-chunk budget $m^{4/3-2\varepsilon_1}$ from Lemma 3 yields the inequality

$$\omega(2/3+2\varepsilon,\ 1/3-\varepsilon_1+\varepsilon_2,\ 1/3-\varepsilon_1+\varepsilon_2) \ \le\ 4/3-2\varepsilon_1, \qquad (2)$$

which is exactly the constraint that forces ε_1 to be strictly positive. The warm-up has its own internal slack parameter, also called ε (paper Assumption 1, $|L_i| \le m^{2/3+2\varepsilon}$); this is distinct from the headline ε of Theorem 6, although the parameter assignment of Lemma 5 sets both to $1/24$ at $\omega = 2$. With the current best rectangular bounds of [1] the warm-up system is feasible at $\varepsilon_1 \approx 0.0420$, $\varepsilon_2 \approx 0.1457$, and $\varepsilon \approx 0.0098$ (paper Appendix B). The closed-form certificate below covers only the conjectured optimum $\omega = 2$, where the constraints reduce to checkable rationals.

The warm-up constraint system consists of inequality (2), its High–High analogue $\omega(1/3+\varepsilon_1, 2/3-\varepsilon_1, 1/3+\varepsilon_1) \le 4/3-2\varepsilon_1$, and the threshold orderings $3\varepsilon_1+2\varepsilon \le \varepsilon_2$, $\varepsilon_1 \le 1/6$, and $\varepsilon_1-\varepsilon_2 \le 1/3$ (corresponding to Equations (5), (2), (6), (7), (8) of [3, §3.4], in that order).

Lemma 5 (Closed-form feasibility at $\omega = 2$). *Under the hypothetical $\omega = 2$, the warm-up constraint system above admits the solution $\varepsilon_1 = 1/24$, $\varepsilon_2 = 5/24$, $\varepsilon = 1/24$.*

Proof. At $\omega = 2$, square matrix multiplication runs in input-size time, and the rectangular exponent collapses to $\omega(a,b,c) = \max(a+b,\ b+c,\ a+c)$, the cost of merely reading the largest pair of factors. We verify each constraint in turn.

Threshold ordering, $\varepsilon_1 \le 1/6$ (paper Eq. (7)): $1/24 < 4/24 = 1/6$. ✓

Threshold ordering, $\varepsilon_1 - \varepsilon_2 \le 1/3$ (paper Eq. (8)): $1/24 - 5/24 = -4/24 < 1/3$. ✓

Sparse–Sparse / Sparse–Dense iteration, $3\varepsilon_1 + 2\varepsilon \le \varepsilon_2$ (paper Eq. (6)): $3 \cdot \frac{1}{24} + 2 \cdot \frac{1}{24} = \frac{5}{24} = \varepsilon_2$, so the constraint holds with equality. ✓

Low–Dense FMM, Eq. (2) (paper Eq. (5)): substitute $a = 2/3+2\varepsilon = 2/3+1/12 = 3/4$ and $b = c = 1/3-\varepsilon_1+\varepsilon_2 = 1/3+4/24 = 1/2$. Then

$$\omega(3/4, 1/2, 1/2) = \max\left(\tfrac{3}{4}+\tfrac{1}{2},\ \tfrac{1}{2}+\tfrac{1}{2},\ \tfrac{3}{4}+\tfrac{1}{2}\right) = \tfrac{5}{4}.$$

The right-hand side is $4/3-2\varepsilon_1 = 4/3-1/12 = 16/12-1/12 = 15/12 = 5/4$, so the constraint is satisfied with equality. ✓

High–High product (paper Eq. (2)), $\omega(1/3+\varepsilon_1, 2/3-\varepsilon_1, 1/3+\varepsilon_1) \le 4/3-2\varepsilon_1$: the dimensions evaluate to $a = c = 9/24 = 3/8$ and $b = 15/24 = 5/8$. At $\omega = 2$,

$$\omega(3/8, 5/8, 3/8) = \max\left(\tfrac{3}{8}+\tfrac{5}{8},\ \tfrac{5}{8}+\tfrac{3}{8},\ \tfrac{3}{8}+\tfrac{3}{8}\right) = 1 \ \le\ 5/4 = 4/3-2\varepsilon_1,$$

verified by the same method as Eq. (2). ✓

All five constraints are satisfied, so the proposed assignment is feasible. □

The verification highlights why the warm-up needs FMM at all: constraints (2) and (paper Eq. (2)) are the only ones in the system that are not trivially linear, and without a sub-cubic ω they would force $\varepsilon_1 = 0$. At $\omega = 2$ they bind tightly, as the equality $\omega(3/4, 1/2, 1/2) = 5/4 = 4/3-2\varepsilon_1$ shows. Appendix B of [3] carries out the analogous verification with the current numerical bounds and confirms a strictly positive slack.

5.6 Pulling it together

With the data structures in Table 2 in place, a query (u, v) with $u \in L_1, v \in L_4$ is answered by case analysis on the classes of u and v [3, Lemma 3.8]. If both endpoints are High, the precomputed triple product $A^{H*} \cdot B_{<i} \cdot C^{*H}$ supplies the answer in $O(1)$. If exactly one endpoint is High or Medium and the other is Medium or Low (the cases HM, ML, HL, MM), we iterate over the at most $2m^{2/3-\varepsilon_1}$ neighbours of the lower-class endpoint and look up two-hop counts in $A^{H*} \cdot B_{<i}$ or $A^{M*} \cdot B_{<i}$, paying $O(m^{2/3-\varepsilon_1})$. The LL case is the most delicate: we iterate over the $2m^{1/3+\varepsilon_1}$ neighbours of v and compute DD, SS, SD contributions from the stored low-class products, with the DS contribution computed symmetrically from u's side using $B_{i,DS} \cdot C^{*L}$, giving $O(m^{1/3+\varepsilon_1})$. Lazy evaluation handles edges of the active chunks B_i, B_{i+1} in $O(m^{2/3-\varepsilon_1})$ additional time. Combining these bounds with Lemma 3 yields the warm-up theorem informally: under Assumptions 1–3, 4-cycles in a fully dynamic 4-layered graph can be counted in worst-case time $O(m^{2/3-\varepsilon_1})$.

5.7 From warm-up to the full theorem (sketch)

The warm-up algorithm leans hard on the fact that A and C are frozen: the running aggregate $A^{L*} \cdot B_{<i,DD} = A^{L*} \cdot B_{<i-1,DD} + A^{L*} \cdot B_{i-1,DD}$ that drives the chunk machinery only makes sense when the left factor does not change. Once A also receives updates, this telescoping breaks down and the FMM step has to be redesigned.

The full algorithm of [3] replaces chunks with *phases* of size $m^{1-\delta}$ for a constant $\delta > 0$. A phase is large enough that, in the time it takes to process its $m^{1-\delta}$ updates, one can multiply two square matrices of dimension $m^{2/3+2\varepsilon}$ using fast matrix multiplication and store all-pairs path counts for the now-frozen previous phase. Queries then combine information from the new phase, the

previous phase (whose matrix products are complete), and the older phases (handled in bulk). The cost analysis in [3, §4] distills to

$$1 - \delta \;\geq\; (2\omega + 1)\,\varepsilon + (\omega - 1)\,\tfrac{2}{3},$$

which has a feasible solution with $\delta > 0$ if and only if $\omega < 5/2 - O(\varepsilon)$. This is the conceptual content of the result: the speedup vanishes long before the cubic barrier, and Strassen's $\omega \approx 2.81$ [13] is therefore *not* sufficient. Only the asymptotically aggressive bounds of [1], which give $\omega \approx 2.371339$, push the exponent below $5/2$ at all. At the conjectural $\omega = 2$ with $\varepsilon = 1/24$, the inequality binds at $\delta = 1/8$ (since $1 - 1/8 = 7/8 = 5/24 + 16/24 = (2 \cdot 2 + 1)/24 + 2/3$).

Theorem 6 (Main result of [3], restated)**.** *There is an algorithm for counting 4-cycles in any fully dynamic graph in worst-case update time $O(m^{2/3-\varepsilon})$ for some constant $\varepsilon > 0$. With current $\omega = 2.371339$, $\varepsilon \approx 0.0098$; with $\omega = 2$, $\varepsilon = 1/24$.*

We restate Theorem 6 verbatim from [3] and do *not* reprove it. Its proof combines the warm-up of this section, the phase machinery sketched above, the removal of Assumption 1 via tiny-vertex handling [3, §6], and the removal of Assumption 2 via vertex-class transitions [3, §7], none of which we reproduce. Our formal contribution in this essay is limited to Theorem 1 (the general-to-layered reduction), Lemma 2 (the $O(n)$ baseline), Theorem 4 (the rectangular FMM step), and Lemma 5 (feasibility of the constraint system at $\omega = 2$).

6 Experiments

6.1 Setup

We describe the experimental design here; full results will appear in the final version of this essay. Two algorithms are compared. Simple-Wedge is a direct implementation of Algorithm 1, maintaining a wedge-count matrix W in dense `numpy` storage and answering queries by an inner product over $N(u)$, achieving $O(n)$ per update by Lemma 2. Naive-Recompute recomputes the global 4-cycle count from scratch at every update via the wedge-sum identity $\#C_4 = \frac{1}{2}\sum_v \binom{W(v)}{2}$, which costs $O(n + m)$ per update and serves as the combinatorial analogue of the static FMM-based counters of [2]; we deliberately avoid an FMM-based recompute baseline, since the asymptotic improvement of [3] over [4] is exactly what this baseline cannot see. The implementation uses Python 3.11 with NumPy on a single workstation (CPU and RAM specifications are TBD). Synthetic inputs are Erdős–Rényi $G(n, p)$ graphs at three densities, with $n \in \{200, 500, 1000, 2000\}$ for both algorithms and an additional $n = 5000$ point for Simple-Wedge only, where Naive-Recompute becomes infeasible in pure Python. We additionally run Simple-Wedge on one small SNAP graph [7], `ca-GrQc` ($n \approx 5$k, $m \approx 14$k). Each input is exercised under two update streams: an insertion-only stream and a mixed 50/50 insert/delete stream applied after a build-up phase.

6.2 Methodology

Each input graph is constructed by streaming the edge set in as a sequence of insertions; we then issue a few hundred no-op queries to warm caches before any timing measurement. Reported per-update times are medians over 1000 subsequent updates, since the median is robust against the GC pauses and JIT-style warm-up effects that distort means in short-lived Python benchmarks. Peak resident memory is captured with `tracemalloc`, summarising the dense n^2 wedge matrix that dominates Simple-Wedge's footprint. Correctness is enforced by a brute-force oracle on small graphs ($n \leq 50$): after every update we recount 4-cycles by enumerating quadruples and abort on any mismatch. This cross-check, applied across every density and update mode in the synthetic suite, is intended to confirm zero output mismatches against the ground truth and so guard the order-of-operations subtleties baked into Algorithm 1.

Fig 3 placeholder

Figure 3: Median per-update time vs. n (log–log) for Simple-Wedge and Naive-Recompute at three densities.

Fig 4 placeholder

Figure 4: Cumulative time vs. number of updates: crossover where dynamic maintenance overtakes recomputation.

Dataset	n	m	Simple-Wedge $\bar{t}$	Recompute $\bar{t}$
`ca-GrQc`	TBD	TBD	TBD	TBD
`email-Enron`	TBD	TBD	TBD	TBD

Table 3: Real-world graph experiments. $\bar{t}$ in milliseconds, median over 1000 updates.

6.3 Results

Figure 3 (forthcoming) plots median per-update time against n on log–log axes, with one curve per (algorithm, density) pair. We expect the Simple-Wedge curves to track a slope-one line, in agreement with the $O(n)$ bound established in Lemma 2: each insertion or deletion touches $|N(u)| + |N(v)| \leq 2n$ entries of the wedge matrix and performs an inner product of the same length. The Naive-Recompute curves are expected to be visibly steeper, reflecting the additional $O(m)$ contribution from rescanning all wedges per update; on the dense $G(n, p)$ regime where $m = \Theta(n^2)$, this should manifest as a slope close to two.

Figure 4 (forthcoming) reports cumulative running time against the number of updates for a fixed graph size, isolating the break-even point at which the constant-factor overhead of maintaining

W is amortised by the savings on each subsequent query. We anticipate that the crossover sits at a small number of updates on dense graphs, where each Naive-Recompute pass is already costly, and shifts right on sparse graphs, where a single recompute is cheap and the dynamic algorithm has less to amortise against. The qualitative prediction is consistent with the discussion in [4] on why maintenance pays off only once update streams grow long.

Table 3 (TBD) reports the measured n, m, median Simple-Wedge update time, and an indicative Naive-Recompute time on the SNAP graph `ca-GrQc` [7], together with peak memory usage of the wedge matrix. The `email-Enron` row is left as a placeholder for a future sparse-wedge implementation; the dense W matrix used here does not scale to that size in pure Python.

6.4 Connection to the theoretical bounds

We stress that this study benchmarks only Simple-Wedge. We do *not* implement the warm-up algorithm of Section 5, whose key step (Theorem 4) requires both chunked maintenance with amortised reconciliation across phases and a competitive rectangular fast matrix multiplication routine. Building such a system to a trustworthy standard is a multi-week engineering effort and falls outside the scope of an exposition essay. Within this scope, the forthcoming numbers are intended to confirm two qualitative predictions of Section 4: that dynamic wedge maintenance becomes dramatically faster than recomputation once the update stream is long enough to amortise the matrix bookkeeping, and that the $O(n)$ worst-case bound of Lemma 2 is loose on sparse graphs, where $m = \Theta(n)$ and only a constant number of wedge entries change, but tight on dense graphs, where almost every entry of W along $N(u) \cup N(v)$ must move. What our experiment cannot speak to is the central practical question raised by [3]: whether the constants hidden inside rectangular FMM [1] permit the asymptotic improvement over [4] to materialise on graph sizes one is likely to encounter in practice. We flag that as an open empirical direction.

7 Conclusion

7.1 What we did

This essay revisited dynamic 4-cycle counting through the lens of incremental view maintenance for the cyclic four-way join. After recasting the general-graph problem as the 4-layered version, Theorem 1 established the reduction by tracking how the layered update protocol turns counted 3-walks into genuine 3-paths. Lemma 2 then gave a complete correctness and complexity analysis of the wedge-maintenance algorithm whose $O(n)$ worst-case update time serves both as a pedagogical baseline and as the algorithm we benchmark. Section 5 carried out the warm-up of [3] in which A and C are frozen, with a complete dimension analysis of the rectangular FMM step (Theorem 4) and a closed-form algebraic certificate (Lemma 5) that the full constraint system admits a strictly positive ε_1 at the conjectural $\omega = 2$. We sketched how phases lift this warm-up to the full $O(m^{2/3-\varepsilon})$ result of Theorem 6, which we restated but did not reprove, and we instantiated and benchmarked the simple algorithm against naive recomputation in Section 6.

7.2 Open questions

Three threads stand out. First, the gap between the $O(m^{2/3-\varepsilon})$ upper bound of [3] and the $\Omega(m^{1/2-\gamma})$ OMv-conditional lower bound of [5]: where on $[1/2, 2/3)$ does the truth lie? The OMv reduction passes through boolean matrix–vector products and so does not rule out further FMM-based speedups, while [3] explicitly notes that a combinatorial $\Omega(m^{2/3})$ lower bound is not yet ruled out either, so the gap could close from either side. Second, IVM-with-FMM as a research direction: this is the first time fast matrix multiplication has been leveraged for join-size estimation in dynamic graphs, and the authors suggest the technique could inspire a new class of IVM algorithms [3]. Where else does FMM break a "natural" barrier in dynamic IVM—other cyclic joins, longer cycles, richer pattern queries? Third, on the empirical side, a faithful implementation of the warm-up with a competitive rectangular FMM routine would test whether the constants of [1] let the asymptotic improvement surface at realistic graph sizes; our benchmarks of the simple algorithm cannot speak to this, and we view it as the natural next step.

References

[1] Josh Alman, Ran Duan, Virginia Vassilevska Williams, Yinzhan Xu, Zixuan Xu, and Renfei Zhou. 2025. More Asymmetry Yields Faster Matrix Multiplication. In *Proceedings of the 2025 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*. 2005–2039.

[2] Noga Alon, Raphael Yuster, and Uri Zwick. 1997. Finding and Counting Given Length Cycles. *Algorithmica* 17, 3 (1997), 209–223.

[3] Sepehr Assadi and Vihan Shah. 2025. An Improved Fully Dynamic Algorithm for Counting 4-Cycles in General Graphs using Fast Matrix Multiplication. arXiv:2504.10748 [cs.DS] PODS 2025, Article 091. https://doi.org/10.1145/3725228.

[4] Kathrin Hanauer, Monika Henzinger, and Qi Cheng Hua. 2022. Fully Dynamic Four-Vertex Subgraph Counting. In *1st Symposium on Algorithmic Foundations of Dynamic Networks (SAND 2022) (LIPIcs, Vol. 221)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 18:1–18:17.

[5] Monika Henzinger, Sebastian Krinninger, Danupon Nanongkai, and Thatchaphol Saranurak. 2015. Unifying and Strengthening Hardness for Dynamic Problems via the Online Matrix-Vector Multiplication Conjecture. In *Proceedings of the Forty-Seventh Annual ACM Symposium on Theory of Computing (STOC)*. 21–30.

[6] Ahmet Kara, Hung Q. Ngo, Milos Nikolic, Dan Olteanu, and Haozhe Zhang. 2020. Maintaining Triangle Queries under Updates. *ACM Transactions on Database Systems* 45, 3 (2020), 11:1–11:46.

[7] Jure Leskovec and Andrej Krevl. 2014. SNAP Datasets: Stanford Large Network Dataset Collection. http://snap.stanford.edu/data.

[8] Ron Milo, Shai Shen-Orr, Shalev Itzkovitz, Nadav Kashtan, Dmitri Chklovskii, and Uri Alon. 2002. Network Motifs: Simple Building Blocks of Complex Networks. *Science* 298, 5594 (2002), 824–827.

[9] Hung Q. Ngo, Ely Porat, Christopher Ré, and Atri Rudra. 2018. Worst-Case Optimal Join Algorithms. *J. ACM* 65, 3 (2018).

[10] Hung Q. Ngo, Christopher Ré, and Atri Rudra. 2014. Skew Strikes Back: New Developments in the Theory of Join Algorithms. *ACM SIGMOD Record* 42, 4 (2014), 5–16.

[11] Pedro Ribeiro, Pedro Paredes, Miguel E. P. Silva, David Aparicio, and Fernando Silva. 2021. A Survey on Subgraph Counting: Concepts, Algorithms, and Applications to Network Motifs and Graphlets. *Comput. Surveys* 54, 2 (2021), 1–36.

[12] Siddhartha Sahu, Amine Mhedhbi, Semih Salihoglu, Jimmy Lin, and M. Tamer Özsu. 2020. The Ubiquity of Large Graphs and Surprising Challenges of Graph Processing: Extended Survey. *The VLDB Journal* 29 (2020), 595–618.

[13] Volker Strassen. 1969. Gaussian Elimination is Not Optimal. *Numer. Math.* 13, 4 (1969), 354–356.